# TideWAL: Timestamp-Integrated Dependency-Closed Parallel WAL for Main-Memory Transaction Processing

Shun Sasaki[1] Takayuki Tanabe[1] Hideyuki Kawashima[1]

**概要**: This paper proposes TideWAL, a dependency-closed parallel WAL protocol for main-memory transaction processing. TideWAL avoids conservative global-prefix durable acknowledgment while preserving recovery safety by acknowledging a transaction only after both its own log record and the durable frontier of its observed dependencies are persistent. In an ERMIA-style implementation, TideWAL reduces pending durable commits from 65,522 to 48 and p99 durable-acknowledgment latency from 131 ms to 512 $\mu$s on YCSB-B at 32 threads, while sustaining 231K durable acknowledgments per second.

## 1. Introduction

### 1.1 Motivation

Transaction processing is a core component of database systems and is used in a wide range of applications that require correctness under concurrent access. When multiple transactions execute concurrently, the database system must preserve a correct database state. Concurrency control provides this isolation guarantee, and serializability has long been used as a fundamental correctness criterion for concurrent transaction execution [1], [2].

Recent main-memory transaction processing systems exploit many-core machines by removing centralized bottlenecks from concurrency control. Systems and protocols such as Silo, ERMIA, TicToc, and other in-memory concurrency-control designs show that transaction execution can scale when validation, timestamp management, and version management are carefully engineered [3], [4], [5], [6]. In timestamp-based engines, including ERMIA-style systems, committed transactions are assigned commit timestamps that are used to order transactions after validation and serializability checks. As a result, the scalability bottleneck of a transaction processing system is not always in concurrency control itself.

When crash durability is required, write-ahead logging (WAL) becomes part of the commit path. WAL is a standard technique for database recovery and is central to classical recovery protocols such as ARIES [7], [8]. A conventional centralized WAL provides a simple and safe durability order: all log records are appended to a single log stream, and transactions are acknowledged according to a durable prefix of that stream. However, this design introduces a centralized serialization point for log insertion and durable acknowledgment.

Parallel WAL (P-WAL) addresses this issue by partitioning log records across multiple WAL shards [9], [10]. By allowing different workers to append to different log streams, P-WAL removes the most obvious shared-log bottleneck. However, parallelizing log insertion does not eliminate the need to reason about durability order. Once the shared WAL mutex is removed, other durability costs become visible, including storage synchronization, durable-point notification, and waiting for a durable condition that is conservative enough to guarantee safe recovery. In asynchronous commit pipelines, these costs appear not only as reduced throughput but also as pending durable commits and increased tail latency.

Therefore, scalable durability for main-memory transaction processing should be evaluated not only by the number of transactions that workers can execute, but also by how quickly the system can safely return durable commit acknowledgments.

[1] 慶應義塾大学
Keio University

### 1.2 Problem

The key problem is that existing P-WAL designs often reintroduce a conservative global ordering mechanism at the durability layer. Even when the concurrency-control layer already assigns a commit timestamp that can serve as the logical order of committed transactions, the WAL layer may separately allocate a global log sequence number (LSN). A transaction is then acknowledged only when a global durable prefix reaches its LSN.

This rule is safe, but it is overly conservative. It couples the durable acknowledgment of transactions that may be independent in the serialization order. If one WAL shard becomes slow, transactions on other shards may be delayed even when they do not depend on any transaction logged by the slow shard. In a bounded asynchronous commit pipeline, this delay may be hidden by allowing many transactions to remain pending. The system may still report high durable-acknowledgment throughput, but only by accumulating a large durable-ack backlog. This backlog increases tail latency and makes the system undesirable for latency-sensitive workloads.

The opposite design point is local-only acknowledgment. In this design, a transaction is acknowledged as soon as its own WAL shard becomes durable. This avoids waiting for unrelated shards, but it is unsafe in general. Suppose transaction $T$ reads a value written by transaction $U$. If $T$'s log record becomes durable before $U$'s log record, local-only acknowledgment may acknowledge $T$ before $U$ is recoverable. A crash at that point can make recovery replay $T$ without the state on which $T$ depended.

Figure 1 illustrates these design points. Global-prefix acknowledgment is safe but may wait for unrelated slow WAL shards. Local-only acknowledgment avoids such waits but can acknowledge a transaction before the transactions it depends on are recoverable. TideWAL instead acknowledges a transaction after its own log record and its dependency-closed durable frontier become durable.

Thus, a durability protocol for P-WAL must avoid both extremes. It should not wait for an unrelated global durable prefix, but it must also avoid acknowledging a transaction before the transactions it depends on are durable.

Figure 2 shows this problem in our ERMIA implementation using YCSB-B [11] at 32 threads. An asynchronous global-LSN-prefix design achieves higher durable-acknowledgment throughput, but it accumulates 65,522 pending commits and increases p99 durable-acknowledgment latency to 131 ms. TideWAL reduces pending commits to 48 and p99 latency to 512 $\mu$s by acknowledging transactions based on dependency-closed durable frontiers. This result shows that high durable-ack throughput can hide substantial durable-acknowledgment backlog.

**図 1** Durable acknowledgment policies in parallel WAL. Global-prefix acknowledgment is safe but conservatively waits for unrelated WAL shards. Local-only acknowledgment avoids unrelated waits but is unsafe when a transaction depends on another non-durable transaction. TideWAL acknowledges a transaction after its own log record and its dependency frontier become durable.

**図 2** Durable acknowledgment behavior on ERMIA with YCSB-B at 32 threads. The asynchronous global-LSN-prefix design achieves higher durable-acknowledgment throughput, but it accumulates 65,522 pending commits and increases p99 durable-acknowledgment latency to 131 ms. TideWAL reduces pending commits to 48 and p99 latency to 512 $\mu$s by using dependency-closed durable acknowledgment.

### 1.3 Approach

This paper proposes *TideWAL*, a timestamp-integrated dependency-closed parallel WAL protocol for main-memory transaction processing. TideWAL is designed for timestamp-based engines such as ERMIA, where the concurrency-control layer already assigns commit timestamps to committed transactions. The goal of TideWAL is not simply to maximize raw throughput, but to return durable commit acknowledgments safely while avoiding

the backlog and tail latency caused by overly conservative global-prefix waiting.

TideWAL is based on three ideas. First, it replaces global-prefix durable acknowledgment with dependency-closed durable acknowledgment. TideWAL acknowledges a transaction only after its own log record and the durable frontier of the transactions on which it depends have become durable. The dependency frontier is derived from actual read/write dependencies observed by the transaction processing engine. This condition preserves recovery safety because every acknowledged transaction is recovered together with the transactions it depends on. At the same time, it avoids waiting for unrelated WAL shards.

Second, TideWAL reuses the commit timestamp as the logical LSN. Existing P-WAL designs may allocate a separate global LSN at the WAL layer even when the transaction engine already has a commit timestamp that represents the logical transaction order. TideWAL removes this duplicated ordering mechanism. WAL shards still maintain physical positions within local log files, but the WAL layer does not allocate an additional global LSN solely for logical recovery ordering.

Third, TideWAL decouples transaction execution from durable acknowledgment by using a worker/flusher/committer pipeline. Worker threads execute transactions, validate them, assign commit timestamps, and enqueue log records. Flusher threads batch and persist shard-local log records. Committer threads observe durable-frontier advances and acknowledge transactions whose dependency conditions are satisfied. This design prevents worker threads from directly blocking on `fdatasync` or on conservative durable-prefix waits.

We implement TideWAL in an ERMIA-style transaction processing engine and evaluate it using YCSB workloads. Our evaluation shows that worker-wait global-prefix WAL severely limits ERMIA's concurrency. For example, on YCSB-B with 32 threads, worker-wait global-prefix WAL achieves 106K durable acknowledgments per second, while an asynchronous global-prefix design reaches 365K durable acknowledgments per second. However, the asynchronous global-prefix design accumulates 65,522 pending commits and increases p99 durable-acknowledgment latency to 131 ms. TideWAL achieves 231K durable acknowledgments per second while reducing pending commits to 48 and p99 latency to 512 $\mu$s. These results show that TideWAL controls durable-acknowledgment backlog and tail latency while preserving recovery safety on sparse-dependency workloads.

We also evaluate the effect of timestamp integration. Under real I/O, the performance difference between a separate-LSN design and TideWAL is small because storage synchronization dominates. Under an I/O-light configuration, however, reusing commit timestamps as logical LSNs improves throughput by 7.4% by eliminating WAL-side global LSN allocation. These results demonstrate that TideWAL removes duplicated ordering between concurrency control and durability, and that the benefit becomes visible when the storage-synchronization bottleneck is weakened.

### 1.4 Organization

The rest of this paper is organized as follows. Section 2 reviews WAL, parallel WAL, timestamp-based concurrency control, and dependency requirements for crash recovery. Section 3 describes the design of TideWAL, including timestamp-integrated logical ordering, dependency-frontier construction, and the worker/flusher/committer pipeline. Section 4 discusses the correctness of dependency-closed durable acknowledgment. Section 5 presents the experimental methodology and evaluates durable-acknowledgment throughput, pending commits, and tail latency. Section 6 discusses related work, including ARIES-style recovery, parallel logging, dependency-based logging, and main-memory recovery systems. Section 7 concludes this paper.

## 2. Background

## 3. Design

## 4. Correctness

## 5. Evaluation

## 6. Related Work

## 7. Conclusion

**謝辞**

**参考文献**

[1] Bernstein, P. A., Hadzilacos, V. and Goodman, N.: *Concurrency Control and Recovery in Database Systems*, Addison-Wesley (1987).

[2] Papadimitriou, C. H.: The serializability of concurrent database updates, *J. ACM*, Vol. 26, No. 4, p. 631–653 (online), DOI: 10.1145/322154.322158 (1979).

[3] Tu, S., Zheng, W., Kohler, E., Liskov, B. and Madden, S.: Speedy transactions in multicore in-memory databases, *Proceedings of the Twenty-Fourth*

*ACM Symposium on Operating Systems Principles*, SOSP '13, New York, NY, USA, Association for Computing Machinery, p. 18–32 (online), DOI: 10.1145/2517349.2522713 (2013).

[4] Kim, K., Wang, T., Johnson, R. and Pandis, I.: ERMIA: Fast Memory-Optimized Database System for Heterogeneous Workloads, *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD '16, New York, NY, USA, Association for Computing Machinery, p. 1675–1687 (online), DOI: 10.1145/2882903.2882905 (2016).

[5] Yu, X., Pavlo, A., Sanchez, D. and Devadas, S.: TicToc: Time Traveling Optimistic Concurrency Control, *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD '16, New York, NY, USA, Association for Computing Machinery, p. 1629–1642 (online), DOI: 10.1145/2882903.2882935 (2016).

[6] Tanabe, T., Hoshino, T., Kawashima, H. and Tatebe, O.: An analysis of concurrency control protocols for in-memory databases with CCBench, *Proc. VLDB Endow.*, Vol. 13, No. 13, p. 3531–3544 (online), DOI: 10.14778/3424573.3424575 (2020).

[7] Haerder, T. and Reuter, A.: Principles of transaction-oriented database recovery, *ACM Comput. Surv.*, Vol. 15, No. 4, p. 287–317 (online), DOI: 10.1145/289.291 (1983).

[8] Mohan, C., Haderle, D., Lindsay, B., Pirahesh, H. and Schwarz, P.: ARIES: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging, *ACM Trans. Database Syst.*, Vol. 17, No. 1, p. 94–162 (online), DOI: 10.1145/128765.128770 (1992).

[9] Xia, Y., Yu, X., Pavlo, A. and Devadas, S.: Taurus: lightweight parallel logging for in-memory database management systems, *Proc. VLDB Endow.*, Vol. 14, No. 2, p. 189–201 (online), DOI: 10.14778/3425879.3425889 (2020).

[10] Haubenschild, M., Sauer, C., Neumann, T. and Leis, V.: Rethinking Logging, Checkpoints, and Recovery for High-Performance Storage Engines, *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, SIGMOD '20, New York, NY, USA, Association for Computing Machinery, p. 877–892 (online), DOI: 10.1145/3318464.3389716 (2020).

[11] Cooper, B. F., Silberstein, A., Tam, E., Ramakrishnan, R. and Sears, R.: Benchmarking cloud serving systems with YCSB, *Proceedings of the 1st ACM Symposium on Cloud Computing*, SoCC '10, New York, NY, USA, Association for Computing Machinery, p. 143–154 (online), DOI: 10.1145/1807128.1807152 (2010).